

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №5
По дисциплине «Современные платформы программирования»
Тема: «Приобрести практические навыки разработки API и баз данных»**

Выполнил:
Студент 3 курса
Группы ПО-12
Боричевский Д. О.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

- 1) Изложение цели работы.
- 2) Задание по лабораторной работе с описанием своего варианта.
- 3) Спецификации ввода-вывода программы.
- 4) Текст программы (кратко).
- 5) Выводы по проделанной работе.

Общее задание

1. Реализовать базу данных из не менее **5 таблиц** на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

- 1) База данных Деканат
- 2) База данных Расписание занятий на факультете

Код программы:

```
"""
```

Single-file FastAPI + SQLAlchemy project for timetable management.

```
"""
```

```
from __future__ import annotations
```

```
from datetime import time
```

```
from typing import Optional, Generator, Iterable
```

```
from fastapi import FastAPI, Depends, HTTPException, status
```

```
from pydantic import BaseModel, Field
```

```
from sqlalchemy import (
```

```
    create_engine,
```

```
    Integer,
```

```
    String,
```

```
    Time,
```

```
    ForeignKey,
```

```
    UniqueConstraint,
```

```
)
```

```
from sqlalchemy.orm import (
```

```
    DeclarativeBase,
```

```
    Mapped,
```

```
    mapped_column,
```

```
    relationship,
```

```
    sessionmaker,
```

```
    Session,
```

```
)
```

```
# -----
```

```
# Database setup
```

```
# -----
```

```
DATABASE_URL = "sqlite:///./schedule.db"
```

```
class Base(DeclarativeBase):
```

```
    """Base class for ORM models."""
```

```
engine = create_engine(
```

```
    DATABASE_URL,
```

```
    connect_args={"check_same_thread": False},
```

```
)
```

```
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

```
def get_db() -> Generator[Session, None, None]:
```

```
    """Provide DB session."""
```

```
    db = SessionLocal()
```

```
    try:
```

```
        yield db
```

```
    finally:
```

```
        db.close()
```

```
# -----
```

```
# Models
```

```
# -----
```

```
class Faculty(Base):
```

```
__tablename__ = "faculties"
```

```
id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
name: Mapped[str] = mapped_column(String(200), unique=True, nullable=False)
```

```
groups: Mapped[list["Group"]] = relationship(
```

```
    back_populates="faculty",
```

```
    cascade="all, delete-orphan",
```

```
)
```

```
class Group(Base):
```

```
    __tablename__ = "groups"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    name: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
```

```
    faculty_id: Mapped[int] = mapped_column(
```

```
        Integer,
```

```
        ForeignKey("faculties.id", ondelete="CASCADE"),
```

```
        nullable=False,
```

```
)
```

```
    faculty: Mapped[Faculty] = relationship(back_populates="groups")
```

```
    lessons: Mapped[list["Lesson"]] = relationship(
```

```
        back_populates="group",
```

```
        cascade="all, delete-orphan",
```

)

```
class Teacher(Base):
```

```
    __tablename__ = "teachers"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    full_name: Mapped[str] = mapped_column(String(200), unique=True, nullable=False)
```

```
    position: Mapped[Optional[str]] = mapped_column(String(100))
```

```
class Classroom(Base):
```

```
    __tablename__ = "classrooms"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    building: Mapped[str] = mapped_column(String(50), nullable=False)
```

```
    room_number: Mapped[str] = mapped_column(String(20), nullable=False)
```

```
    __table_args__ = (
```

```
        UniqueConstraint("building", "room_number", name="uq_building_room"),
```

```
)
```

```
class Course(Base):
```

```
    __tablename__ = "courses"
```

```
id: Mapped[int] = mapped_column(Integer, primary_key=True)

title: Mapped[str] = mapped_column(String(200), unique=True, nullable=False)

hours_per_week: Mapped[int] = mapped_column(Integer, nullable=False, default=2)
```

```
class Lesson(Base):
```

```
    __tablename__ = "lessons"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    weekday: Mapped[int] = mapped_column(Integer, nullable=False)

    start_time: Mapped[time] = mapped_column(Time, nullable=False)

    end_time: Mapped[time] = mapped_column(Time, nullable=False)
```

```
    group_id: Mapped[int] = mapped_column(

        Integer,

        ForeignKey("groups.id", ondelete="CASCADE"),

        nullable=False,

    )
```

```
    course_id: Mapped[int] = mapped_column(

        Integer,

        ForeignKey("courses.id", ondelete="CASCADE"),

        nullable=False,

    )
```

```
    teacher_id: Mapped[int] = mapped_column(

        Integer,

        ForeignKey("teachers.id", ondelete="SET NULL"),
```

```

        nullable=False,
    )

    classroom_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("classrooms.id", ondelete="SET NULL"),
        nullable=False,
    )

    group: Mapped[Group] = relationship(back_populates="lessons")

```

```

__table_args__ = (
    UniqueConstraint("weekday", "start_time", "group_id", name="uq_group_time"),
)

```

```

# -----

```

```

# Schemas

```

```

# -----

```

```

class FacultyCreate(BaseModel):
    name: str = Field(..., max_length=200)

```

```

class FacultyRead(FacultyCreate):
    id: int

```

```
class Config:  
  
    from_attributes = True
```

```
class GroupCreate(BaseModel):  
  
    name: str  
  
    faculty_id: int
```

```
class GroupRead(GroupCreate):  
  
    id: int
```

```
class Config:  
  
    from_attributes = True
```

```
class TeacherCreate(BaseModel):  
  
    full_name: str  
  
    position: Optional[str] = None
```

```
class TeacherRead(TeacherCreate):  
  
    id: int
```

```
class Config:  
  
    from_attributes = True
```

```
class ClassroomCreate(BaseModel):
```

```
    building: str
```

```
    room_number: str
```

```
class ClassroomRead(ClassroomCreate):
```

```
    id: int
```

```
class Config:
```

```
    from_attributes = True
```

```
class CourseCreate(BaseModel):
```

```
    title: str
```

```
    hours_per_week: int = 2
```

```
class CourseRead(CourseCreate):
```

```
    id: int
```

```
class Config:
```

```
    from_attributes = True
```

```
class LessonCreate(BaseModel):
```

```
    weekday: int = Field(ge=1, le=7)
```

```
    start_time: time
```

```
    end_time: time
```

```
    group_id: int
```

```
    course_id: int
```

```
    teacher_id: int
```

```
    classroom_id: int
```

```
class LessonUpdate(BaseModel):
```

```
    weekday: Optional[int] = Field(default=None, ge=1, le=7)
```

```
    start_time: Optional[time] = None
```

```
    end_time: Optional[time] = None
```

```
    group_id: Optional[int] = None
```

```
    course_id: Optional[int] = None
```

```
    teacher_id: Optional[int] = None
```

```
    classroom_id: Optional[int] = None
```

```
class LessonRead(LessonCreate):
```

```
    id: int
```

```
class Config:
```

```
    from_attributes = True
```

```
# -----
```

```
# FastAPI app
```

```
# -----
```

```
app = FastAPI(title="Schedule API", version="1.0")
```

```
Base.metadata.create_all(bind=engine)
```

```
# -----
```

```
# CRUD Endpoints
```

```
# -----
```

```
@app.post("/faculties", response_model=FacultyRead, status_code=201)
```

```
def create_faculty(faculty: FacultyCreate, db: Session = Depends(get_db)) -> Faculty:
```

```
    obj = Faculty(name=faculty.name)
```

```
    db.add(obj)
```

```
    db.commit()
```

```
    db.refresh(obj)
```

```
    return obj
```

```
@app.get("/faculties", response_model=list[FacultyRead])
```

```
def get_faculties(db: Session = Depends(get_db)) -> Iterable[Faculty]:
```

```
    return db.query(Faculty).all()
```

```
@app.post("/groups", response_model=GroupRead, status_code=201)
```

```
def create_group(group: GroupCreate, db: Session = Depends(get_db)) -> Group:
```

```
    obj = Group(**group.model_dump())
```

```
    db.add(obj)
```

```
    db.commit()
```

```
    db.refresh(obj)
```

```
    return obj
```

```
@app.get("/groups", response_model=list[GroupRead])
```

```
def get_groups(db: Session = Depends(get_db)) -> Iterable[Group]:
```

```
    return db.query(Group).all()
```

```
@app.post("/teachers", response_model=TeacherRead, status_code=201)
```

```
def create_teacher(teacher: TeacherCreate, db: Session = Depends(get_db)) -> Teacher:
```

```
    obj = Teacher(**teacher.model_dump())
```

```
    db.add(obj)
```

```
    db.commit()
```

```
    db.refresh(obj)
```

```
    return obj
```

```
@app.get("/teachers", response_model=list[TeacherRead])
```

```
def get_teachers(db: Session = Depends(get_db)) -> Iterable[Teacher]:  
    return db.query(Teacher).all()
```

```
@app.post("/classrooms", response_model=ClassroomRead, status_code=201)
```

```
def create_classroom(  
    classroom: ClassroomCreate,  
    db: Session = Depends(get_db),  
) -> Classroom:  
    obj = Classroom(**classroom.model_dump())  
    db.add(obj)  
    db.commit()  
    db.refresh(obj)  
    return obj
```

```
@app.get("/classrooms", response_model=list[ClassroomRead])
```

```
def get_classrooms(db: Session = Depends(get_db)) -> Iterable[Classroom]:  
    return db.query(Classroom).all()
```

```
@app.post("/courses", response_model=CourseRead, status_code=201)
```

```
def create_course(course: CourseCreate, db: Session = Depends(get_db)) -> Course:  
    obj = Course(**course.model_dump())  
    db.add(obj)  
    db.commit()
```

```
db.refresh(obj)
```

```
return obj
```

```
@app.get("/courses", response_model=list[CourseRead])
```

```
def get_courses(db: Session = Depends(get_db)) -> Iterable[Course]:
```

```
    return db.query(Course).all()
```

```
@app.post("/lessons", response_model=LessonRead, status_code=201)
```

```
def create_lesson(lesson: LessonCreate, db: Session = Depends(get_db)) -> Lesson:
```

```
    obj = Lesson(**lesson.model_dump())
```

```
    db.add(obj)
```

```
    db.commit()
```

```
    db.refresh(obj)
```

```
    return obj
```

```
@app.get("/lessons", response_model=list[LessonRead])
```

```
def get_lessons(db: Session = Depends(get_db)) -> Iterable[Lesson]:
```

```
    return db.query(Lesson).all()
```

```
@app.get("/lessons/{lesson_id}", response_model=LessonRead)
```

```
def get_lesson(lesson_id: int, db: Session = Depends(get_db)) -> Lesson:
```

```
    obj = db.query(Lesson).filter(Lesson.id == lesson_id).first()
```

if obj is None:

raise HTTPException(404, "Lesson not found")

return obj

@app.put("/lessons/{lesson_id}", response_model=LessonRead)

def update_lesson(

lesson_id: int,

update: LessonUpdate,

db: Session = Depends(get_db),

) -> Lesson:

obj = db.query(Lesson).filter(Lesson.id == lesson_id).first()

if obj is None:

raise HTTPException(404, "Lesson not found")

for field, value in update.model_dump(exclude_unset=True).items():

setattr(obj, field, value)

db.commit()

db.refresh(obj)

return obj

@app.delete("/lessons/{lesson_id}", status_code=204)

def delete_lesson(lesson_id: int, db: Session = Depends(get_db)) -> None:

obj = db.query(Lesson).filter(Lesson.id == lesson_id).first()

if obj is None:

raise HTTPException(404, "Lesson not found")

db.delete(obj)

db.commit()